Capabilities of ChatGPT for Code Analysis: An Empirical Study

***Abstract*—ChatGPT, a disruptive innovation, has shown great potential for reforming software engineering (SE) by demonstrating impressive performance on different software engineering tasks. Although it has superior performance, but we cannot figure out whether the superior performance of ChatGPT is from its ability to comprehend program semantics well or only copy-paste the seen data. Moreover, the extent that ChatGPT comprehends program semantics is also unknown. To address these limitations, in this paper, we conduct a comprehensive empirical study to explore the capabilities of ChatGPT in comprehending program syntax rules, static behaviors, and dynamic behaviors through 9 different tasks on 2,327 code samples. In particular, we design the task of AST generation and expression matching to investigate whether ChatGPT is able to understand code syntax rules. Furthermore, we analyze ChatGPT's capabilities in understanding code static behaviors e.g., control flows, function invocations, and data dependencies. We further use mutant testing detection and flaky test reasoning to approximate code dynamic behaviors. Extensive experiments have confirmed that ChatGPT is more powerful in understanding code syntax and has certain abilities to analyze code static behaviors but is very limited to approximate code dynamic behaviors. We believe our findings provide insights into the performance of ChatGPT on SE tasks and hope to guide the follow-up researchers in effectively using ChatGPT.**

I. Introduction

ChatGPT [1], released by OpenAI in November 2022, has become one of the most attention-grabbing achievements in the era of AI-Generated Content (AIGC). It is a conversational AI system trained from a foundation language model (a large language model) from the GPT-3.5 series with Reinforcement Learning from Human Feedback (RLHF) [2]–[4]. The massive learned knowledge in the GPT-3.5 series and the elegant fine-tuning process by RLHF enable ChatGPT to generate high-quality responses to user questions in various domains. Its ability to comprehend context, align instructions, and produce coherent content makes it excellent at multiple tasks such as bilingual translation [5], conversation generation [6] and article summarization [7].

Furthermore, ChatGPT also has attracted widespread attention from software engineering as it exhibits excellent capability in software development [8]. It has been widely used in different stages of software development. For example, it can be used in generating code snippets that satisfy the natural language requirements according to the official report [9] by OpenAI. Researchers from the SE community started to explore how to use ChatGPT in SE tasks, for example, Xia et al. [10] proposed ChatRepair, which aims to interact with ChatGPT to perform automated program repair in a conversational style. Tian et al. [11] conducted an empirical study to discuss the capability of ChatGPT for code generation, program repair, and code summarization. However, although ChatGPT is widely used and discussed in software engineering, a deep and systematic analysis of ChatGPT's capabilities for code semantics understanding is vital and worthy of in-depth study.

Program semantics is the essence of a program, and automated learning program semantics requires the model to comprehend program syntax rules, static behaviors (e.g., data dependencies), and dynamic behaviors (e.g., execution paths). Hence, program semantics can be regarded as the core for various code-related tasks such as code summarization, code search, and program repair. Pioneering researchers explored the way from different dimensions to accurately learn program semantics such as learning from program structures [12], [13], combining with external knowledge [14], [15], or selecting more powerful neural networks [16], [17]. With the continuous development of techniques, large language models (LLMs) are adopted and they achieve significant progress across various tasks. When the software developer uses these LLMs as the tools of the programming assistants for their coding tasks, the stunning performance often leads the user to believe that the model has comprehended the program semantics well and the model is able to produce accurate results. However, *can these LLMs really comprehend program semantics?* Some previous works have confirmed that these LLMs are prone to suffer from adversarial attacks [18], [19]. Some simple modifications to the input can mislead the model to produce unexpected outputs. It is unclear whether these LLMs are able to comprehend program semantics or only copy-paste similar content from the training samples. Moreover, if these LLMs have a certain capability to comprehend program semantics, to what extent can they comprehend the semantics is also unknown.

To address the aforementioned challenges, in this paper, we conduct a progressive analysis to explore the capability of ChatGPT to comprehend program semantics in terms of understanding program syntax, static behaviors, and dynamic behaviors. To achieve this, we design a set of code-related tasks (**9** different tasks) on **2,327** code samples. Specifically, for code syntax understanding, we design two tasks, Abstract Syntax Tree (AST) generation and expression matching to find out whether ChatGPT can comprehend program syntax. Besides, we design five tasks including Control Flow Graph (CFG) generation, Call Graph generation, data dependency analysis, taint analysis, and pointer analysis to explore whether ChatGPT is able to statically approximate program behavior

similar to other conventional static analysis tools [20], [21]. Based on syntax understanding and static understanding, we further propose two more challenging tasks: equivalent mutant detection and flaky test reasoning to analyze the capability of ChatGPT in dynamically analyzing program behaviors. Through our comprehensive analysis, we found that (1) ChatGPT is more powerful in understanding code syntax. It is able to understand the syntax role of tokens in the code, thus ChatGPT can act as an AST parser. (2) ChatGPT has certain abilities to analyze the code static behaviors and it can act as a beginner in static analysis. (3) ChatGPT is limited to approximate code dynamic behaviors and thus has a poor performance on mutant detection and flaky test reasoning.

Overall, the main contributions of our paper are summarized as follows:

1) We conduct a comprehensive study from different aspects to explore the capability of ChatGPT for code analysis. We are the first to explore ChatGPT's capability in understanding code syntax, static behaviors, and dynamic behaviors. We have made our code and data public on our website [22].
2) We analyze ChatGPT on understanding code syntax, code static semantic structures and code dynamic behaviors through diverse tasks. Our study suggests that ChatGPT is capable to comprehend code syntax rules and it has certain abilities to understand code static behaviors but fails to comprehend dynamic behaviors.

II. Motivation

With the strong capabilities of GPT-series models in coding, such as Copilot [1] and AlphaCode [2], more developers utilize them to recommend the needed code snippets in the daily software development. The recommended content of these tools usually provides users with a great experience. More recently, some tools rely on ChatGPT for program repair [10], [23] are proposed. The accurate and satisfactory results produced by these tools seem to be proving to users that these LLMs are able to learn program semantics well. However, *do they really learn program semantics rather than copy-pasting from seen samples?* Since the training samples used to train ChatGPT are from the Internet across the world by the end of 2021. Hence, a reasonable assumption is that the queried content is a part of the training samples and ChatGPT only serves as a content distributor to finish copy-paste operation.

To make clear whether ChatGPT is able to comprehend program semantics or not, a preliminary experiment is conducted for exploration in our study. Concretely, we use ChatGPT playground [3] with *temperature* of value 0 to perform the conversation. As shown in Fig. 1, there is a buggy function "bucketsort" obtained from QuixBugs [4]. The Bucketsort algorithm requires splitting the array (i.e., "arr" in this function) into a number of buckets (i.e., "counts") and then sorting

Does this program have a bug? How to fix it?

```
def bucketsort(arr, k ):
    counts = [0] * k
    for x in arr:
        counts[x] += 1
    sorted_arr = []
    for i, count in enumerate(arr):
        sorted_arr.extend([i] * count )
    return sorted_arr
```

Yes, this program has a bug. The bug is that the variable "counts" is not being used in the second for loop. To fix this bug, the variable "counts" should be used in the second for loop instead of "arr". The corrected code should look like this:

```
def bucketsort(arr, k):
    counts = [0] * k
    for x in arr:
        counts[x] += 1
    sorted_arr = []
    for i, count in enumerate(counts):
        sorted_arr.extend([i] * count)
    return sorted_arr
```

Fig. 1: A buggy function of Bucketsort from QuixBugs.

each bucket individually. Hence, we can find that the correct version to fix this bug function is to replace the variable "arr" in the second loop with the variable "counts". According to an analysis from Sobania et al. [23], ChatGPT is able to automatically fix this bug and it is shown in Fig. 1. Through this example, it seems that ChatGPT correctly comprehends this function semantics and finishes a correct repair.

However, a simple mutation while maintaining the original program semantics can lead ChatGPT to produce the incorrect results. As shown in Fig. 2, if we change the variable "arr" to another variable name "ccounts" while keeping the other content the same as the former example, ChatGPT cannot fix this buggy function. Hence, the conclusion from both examples is inconsistent. If we just investigate the first example, it seems that ChatGPT is able to comprehend program semantics and thus successfully fix this buggy function. However, if we further consider the second example, the conclusion is invalid.

As ChatGPT is becoming popular in software engineering (SE) and it plays a significant role in software development, the comprehension of its effectiveness is urgent and significant. Furthermore, understanding the capabilities and limitations of ChatGPT for code analysis are important and it will ensure that the SE researchers and software developers can use ChatGPT correctly and reasonably for their tasks. To address this challenge, in this paper, we provide a systematic and comprehensive study to investigate the capabilities of ChatGPT for code analysis i.e., what it can do, and what its limitations are.

III. Study Design

ChatGPT is an AI product, researchers do not have access to its internal mechanisms, including its training strategies and

[1] https://github.com/features/copilot
[2] https://www.deepmind.com/blog/competitive-programming-with-alphacode
[3] https://platform.openai.com/playground
[4] https://github.com/jkoppel/QuixBugs/

Does this program have a bug? How to fix it?

```
def bucketsort(ccounts, k ):
    counts = [0] * k
    for x in ccounts :
        counts[x] += 1
    sorted_arr = []
    for i, count in enumerate(ccounts):
        sorted_arr.extend([i] * count )
    return sorted_arr
```

Yes, this program has a bug. The variable "ccounts" is used in the first line of the program, but it is not defined anywhere. To fix this bug, the variable "ccounts" should be defined and given a value.

Fig. 2: An semantic equivalent version of the buggy function from Fig. 1.

data. This lack of transparency prevents the use of classical code model probing analysis approaches [24]–[26] for ChatGPT. To address this limitation, we begin by examining the abilities that AI models must possess to effectively tackle SE tasks. Generally, through a broad literature review of code modeling works with neural networks, we can categorize existing code models into three different groups. First, some code models [16], [27]–[31] are designed to learn code syntax by incorporating abstract syntax tree (AST) information to assist in different SE tasks. Second, researchers also explore the use of control flow and data flow information to enhance code models [32]–[36]. Third, there are some works [37]–[39] leverage code execution to learn code representations that can address SE tasks related to code semantics.

Based on our literature review, in this work, we conduct a comprehensive study to explore the capabilities of ChatGPT for code analysis and answer the following three research questions (RQs):

- ***RQ1: Can ChatGPT understand code syntax well?***
- ***RQ2: Can ChatGPT understand code static behaviors?***
- ***RQ3: Can ChatGPT understand code dynamic behaviors?***

Fig. 3 demonstrate how we estimate ChatGPT on Software Engineering. The fundamental abilities required to competently perform Software Engineering tasks consist of understanding the syntax of code, understanding the code static behaviors, and understanding the code dynamic behaviors. For syntax understanding (RQ1), we first check if ChatGPT can understand code syntax structure (AST) and then we design a task to require ChatGPT to search mathematics expressions that require highly understanding of the syntax roles of tokens. AST is the core structure in code analysis and represents the programming language syntax. We consider using the expression matching task because the expressions are concise for easy analysis and all the expressions tokens have the syntax roles. Literally matching without understanding these syntax roles leads to poor performance. For static understanding (RQ2), as the control flow of code estimates the order of statement execution, which is considered the first step in program analysis. Hence we begin with CFG and require ChatGPT to construct CFG. Call graph (CG) is useful to analyze large projects by providing calling context. We also require ChatGPT to construct CG. Then, we conduct data dependency, taint analysis, and pointer analysis. These three tasks reflect data flow information that is crucial for code analysis. For dynamic understanding (RQ3), we use equivalent mutant detection and flaky test reasoning. Equivalent mutant refers to a changed program having the same behavior as the original code. It is directly related to code dynamic behavior. Flaky test reasoning entails explaining why a test sometimes passes and sometimes fails with the same setting. Dynamic behaviors of flaky tests can test how ChatGPT understands program execution with concrete inputs. By investigating ChatGPT's capabilities in these tasks, we can make clear what it can do, as well as what it cannot do. This understanding can help us figure out how to use ChatGPT in Software Engineering.

A. Code Syntax Understanding (RQ1)

Code syntax refers to the set of rules that define valid combinations of symbols in a given programming language. Abstract Syntax Tree (AST) is a data structure that represents code syntax. In this study, we aim to investigate whether ChatGPT can understand code syntax using AST.

1) AST Generation: We begin our evaluation by assessing whether ChatGPT can recognize code AST structures. We prompt ChatGPT to parse code into an AST, and then we compare these ASTs with those generated by programming language AST parsers to determine their meaningfulness. The ability to comprehend ASTs is fundamental for code models, as the tokens in code serve distinct syntax roles. Understanding code syntax is crucial for addressing certain SE tasks, such as generating syntax-correct code.

2) Expression Matching: This task means that we find a similar expression with target mathematics expression from the input code. The matched expression should have almost the same operators as the target expression. Fig. 4 presents an example of this task, in which we try to find a similar expression with "base_borrow_rate+utilization_rate*slope1". To accomplish this task, one must understand the operators, operands, and their order in the mathematical expressions defined by programming language syntax rules. Without understanding the syntax role of tokens, finding similar expressions is not feasible. For instance, "base_borrow_rate-utilization_rate+slope1" may be incorrectly identified as a more similar expression to "base_borrow_rate+utilization_rate*slope1" than to "rate+ur*s1", if the operators "+" and "*" are not recognized. Although the former has similar variable names to the target expression, greater attention should be paid to the syntax-role-operator tokens because they decides the code behaviors. We evaluate whether ChatGPT can leverage its comprehension of code syntax structure to perform an expression matching task.

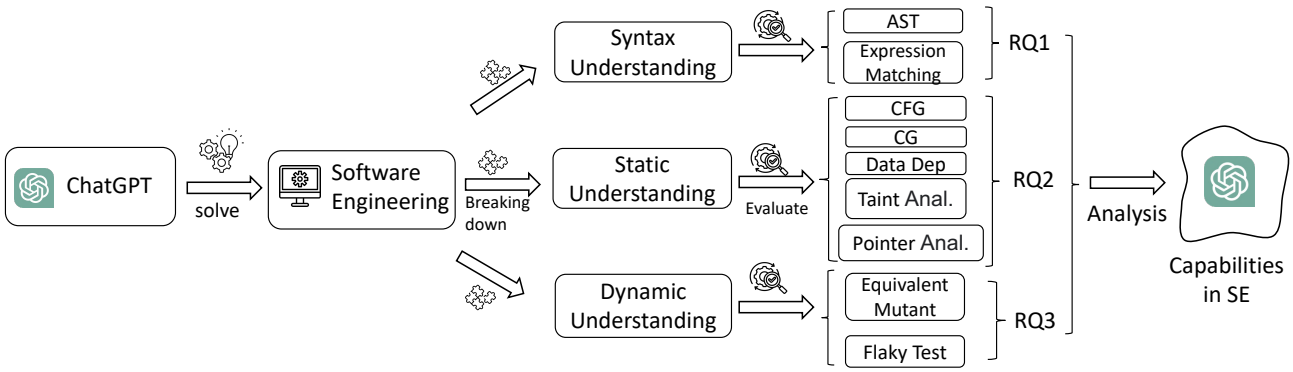


Fig. 3: The overview of our study.

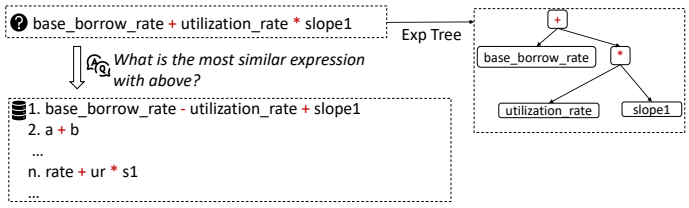


Fig. 4: An example of the task of Expression Matching.

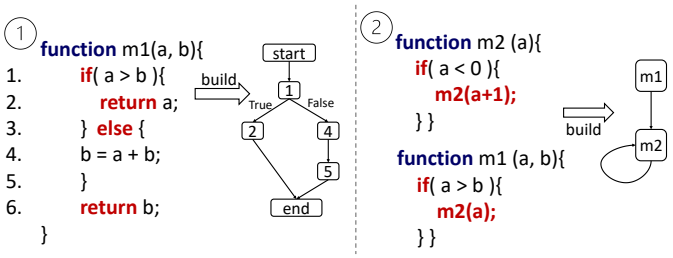


Fig. 5: (1) Code Control Flow Graph and (2) Code Call Graph.

B. Code Static Behavior Understanding (RQ2)

Static analysis provides fundamental knowledge for solving various Software Engineering tasks, including bug detection and test generation. Through static analysis, it is possible to identify the relationships among different components in a program, such as the data flow of variables. The objective of this study is to assess the performance of ChatGPT in code static behavior analysis, focusing on five types of static behaviors, control flow graph (CFG), call graph (CG), data dependency, taint analysis and pointer analysis.

1) Control Flow Graph (CFG) Analysis: Control flow graph (CFG) analysis is typically the first step in program analysis as it estimates the order of statement execution. To achieve this, we prompt ChatGPT to construct the CFG for a given input code. Fig. 5 provides an example ① of this process. Understanding the CFG is critical for code models to identify relationships among statements. CFG is a core code structure in static analysis and is extensively employed in Software Engineering to address various tasks such as vulnerability detection, code optimization, and program analysis [40]–[42].

2) Call Graph (CG) Analysis: The Call Graph is a diagram that depicts the invocation relationship among functions in a program. It is extensively employed in Software Engineering such as interprocedural program analysis. Fig. 5 presents an example ② of a call graph with two methods. We prompt ChatGPT to construct the call graph from the input code. The call graph can also help developers understand the program. To model code by neural networks, it is essential to understand a complex program. Therefore, comprehending the call graph is significant as it provides insights into the function relationships in the code.

3) Data Dependency: Fig. 6 provides an example ① in which "d" is data-dependent on "a". We prompt ChatGPT to determine whether two given variables are data-dependent in the code. Data dependency analysis is a powerful technique for code analysis and optimization as it can reveal data relationships among different variables in a program. Data dependency illustrates how data are propagated in the program, and it is extremely useful for code models to solve SE tasks such as vulnerability detection.

4) Taint Analysis: In this study, we employ taint analysis to detect whether a variable can be tainted by an external source. Fig. 6 illustrates an example ② in which the variable "a" can be overwritten by "source_taint" via the storage variable "r". This task necessitates the reasoning ability of AI models based on data dependency analysis.

5) Pointer Analysis: Pointer analysis is a challenging problem in Software Engineering that analyzes the storage positions referred to by pointers. Fig. 6 illustrates an example ③ of pointer analysis. The pointer "x" can potentially point to either "c" or "d". Pointer analysis is extensively used to detect vulnerabilities such as memory leakage. We prompt ChatGPT to infer the referents of pointers. This task requires artificial intelligence models to comprehend the code syntax and semantics in-depth.

C. Code Dynamic Behavior Understanding (RQ3)

Code syntax and static information are typically the initial inputs required to solve more complex Software Engineering tasks that involve code dynamic behavior. We will evaluate the ability of ChatGPT in understanding code dynamics through

```
function f (a, b){
// Data Dependency
    c = a ;
    d = 0;
  if( b < 0 ){
      d = c;
  }
    return d;
}
```

①

```
int main(){
// Pointer Analysis
  int c = m();
  int d = f(c);
  int* x = d>0?&c:&d;
}
```

③

```
contract Reference{
  // Taint Analysis
  struct Ref{
      uint val;
  }
  Ref a;
  Ref b;
  function set(uint source_taint)
  public
  {
      Ref storage r = a;
      if(true){
        r = b;
      }
      r.val = source_taint;
  }
}
```

②

Fig. 6: (1) Data Dependency Example, (2) Taint Analysis Example and (3) Pointer Analysis.

two widely known tasks: equivalent mutant detection and flaky test reasoning.

1) Equivalent Mutant Detection: Detecting equivalent mutants is a critical problem in Mutation Testing [43], which ensures that the mutanted code remains functionally equivalent after introducing a random change. Mutation Testing is a technology used to estimate and improve the quality of the test suite by observing how many mutants are detected by the test cases and generating new test cases to identify these injected bugs. Any code change is highly possible to change code dynamic behaviors. This task is consistent with the goal of RQ3. We prompt ChatGPT to determine whether a mutant is equivalent to the original code. Fig. 7 presents an example of an equivalent mutant ①. In this example, the variable 'b' in 'a+b' has been changed to 'b–', but this modification does not affect the result of 'a+b'.

2) Flaky Test Reasoning: Flaky test is one other challenging problem in Software Engineering. Flaky test means the output inconsistency of one test when running multiple times. Thirty flaky reasons are summarized [44]. Flaky tests are usually caused by some undetermined functions, the environment state and the execution schedule. Fig. 7 presents one flaky example ② due to randomness. This task is complementary to the equivalent mutant detection task because it does not change code but can behave differently with the same inputs. We prompt ChatGPT to tell the reason why one test is flaky.

D. Prompt Design

GPT-based models utilize the prompt-based learning paradigm [45]. The design of the prompt can significantly impact the performance of the model. To design better prompts, we first employ ChatGPT to optimize our initial prompts and then summarize the generated prompts. Specifically, we prompt ChatGPT with the message "*Act as a prompt optimizer and optimize the following prompt for [TASK DESCRIPTION].*

①

```
function f (a, b){
    c = a + b ;
    d = 0;
    if( b < 0 ){
        d = c;
    }
    return d;
}
```

Eq. Mutant ⟹

```
function f (a, b){
    c = a + ( b-- ) ;
    d = 0;
    if( b < 0 ){
        d = c;
    }
    return d;
}
```

②

```
function test_h (){
    n = random() ;
    a = 10;
    assert h(a+n), "Test Failed"
}
```

Fig. 7: (1) Equivalent Mutant Example and (2) Flaky Test Reasoning Example.

The prompt is [PROMPT]" to help us generate prompts. For each task, we have multiple prompt drafts, and then we manually evaluate them using some task data to observe their difference so that we can obtain high-quality prompts. Finally, according to the obtained experience from trials, we summarize our prompt templates as *role prompt* and *instruction prompt*.

Role prompt assigns a specific role to ChatGPT, supplying task context for the model to generate the desired output effectively. Its template is shown below,

```
You are [ROLE] for [LANG]. [TASK
DESCRIPTION]. [OUTPUT FORMAT]. The
input is [INPUT].
```

where the placeholder [ROLE] denotes the specific role assigned to ChatGPT. We define six roles: AST parser, expression tree matcher, control flow graph analyzer, call graph analyzer, code static analyzer, and pointer analyzer. [LANG] refers to the used programming language for the analyzed code. [TASK DESCRIPTION] outlines the expected task for ChatGPT to perform. [OUTPUT FORMAT] provides the output specification. [INPUT] serves as the placeholder for the code under analysis.

Instruction prompt does not assign a specific role to ChatGPT, instead, they provide a command. These prompts are typically useful for tasks involving multiple roles or those without any applicable roles. The template for the instruction prompt is defined as follows:

```
Please analyze [LANG]. [DOMAIN
KNOWLEDGE]. Here are some examples
[EXAMPLE CODE]. Please identify
if [TASK DESCRIPTION]. [OUTPUT
FORMAT]. The input is [INPUT].
```

where [LANG] specifies the used programming language for the analyzed code. [DOMAIN KNOWLEDGE] explains the domain knowledge relevant to the task. [EXAMPLE CODE] provides sample code related to domain knowledge for the task demonstration. [TASK DESCRIPTION] describes the task instruction. [OUTPUT FORMAT] outlines the output specification. [INPUT] serves as the placeholder for the code under analysis. In our study, we employ the role prompt for RQ1 and RQ2. However, for RQ3, we utilize the instruction prompt because the tasks in RQ3 are not suited for completion by a single role.

You are a C Abstract Syntax Tree (AST) parser. I will give you a C code file. You give me its AST in Json format. Each AST node only has three attributes, children, type and value.

The input file is
```
[INPUT_CODE]
```

Role Prompt for AST task

Please analyze the two following provided code files in C or Java. Identify if they are semantically equal. 'Semantically equal' means two codes have the same meaning, that they have the same output given the same input. Here are three semantically equal examples:
The first example pair is
```
Code 1
  double f(double M, double x) {
    x = (M + x) / 2;
    return x;
  }
```

Instruction Prompt for Equal Mutant

```
Mutant Code 1
  double f(double M, double x) {
    x = (M + x++ ) / 2;
    return x;
  }
```

Yes. The two codes are semantically euqal because `M + x++` first does `M + x` and then `x++`. Therefore, `(M + x) / 2` is the same with `(M + x++) / 2`.

[More examples]

Please identify if the two following codes are semantically equal. Please only answer `yes` or `no`. `yes` means they are semantically equal. `no` means they are not.
Input :
```Code
[INPUT_CODE]
```

Fig. 8: Prompt examples for the task of AST generation and equivalent mutant detection.

Fig. 8 demonstrate two examples of the prompts used for AST generation and equivalent mutant detection respectively. We present the prompts for other tasks on our website [22].

IV. Evaluation Setup

As ChatGPT has been trained on the data of the Internet as of the end of 2021, hence to avoid the risk of data leakage, we utilized new data generated by program tools or recently created datasets. We also created a new dataset for Expression Matching using four decentralized finance projects and their whitepapers. The tasks and datasets utilized in our study are summarized in Table I. We utilized the ChatGPT API for most tasks, with the exception of AST, CFG, and CG generation tasks. For these tasks, we manually conducted them on the ChatGPT website.

A. Dataset and Evaluation Metrics

1) AST Generation: Since programs typically consist of sequential statements, if-else structures, loop structures, try-catch, and switch structures, we used small programs that implement these structures. Through the analysis of these basic syntax structures, we can effectively assess the ability of ChatGPT to understand code syntax. Additionally, AST can be redundant, and even a simple program can have a large AST with many nodes, which may lead to output truncation for ChatGPT due to its maximum token handling limitations. We considered 32 programs in Python, Java, C, and Solidity. To assess ChatGPT's ability to output AST in JSON format, we visualized its outputs for manual evaluation. During the evaluation, we referred to the AST formats from tree-sitter [5] parser. We classified ChatGPT's output as reasonable or not by analyzing the entire structure with a tolerance for some minor issues such as missing trivial leaf nodes. If the output was deemed acceptable, we labeled it as 'Yes'. If the output provided a wrong structure with incorrect edges and nodes, we labeled it as 'No'. In the end, we count how many programs are handled reasonably and record the issues we find.

2) Expression Matching: This dataset was collected to address a real problem in the blockchain economy. Decentralized finance systems often detail their reward mechanisms in the project whitepaper and make promises that the mechanism is implemented in the project. To create this dataset, we randomly selected four real projects from Ethereum [6], namely ALPHA, BETA, BiFi, and XEN. We implemented 32 reward computing equations strictly based on their whitepapers and then prompted ChatGPT to find similar expressions in the target Solidity projects. For each query, we fed the target code to ChatGPT as comprehensively as possible and then checked if the similar expression was in the top-5, top-10, and top-20 output expressions. In this task, we used CodeBERT [16] as the baseline. We extracted vector representations using CodeBERT and then computed the cosine similarity between our implemented expressions and the expressions in the project.

3) Control Flow Graph (CFG) Generation: Control flow graphs (CFGs) comprise sequence blocks, branch blocks, and loop blocks. To complete this task, we used the dataset from the AST generation task. We visualized the outputs of ChatGPT for manual evaluation, following the same process as the AST task. We labeled the generated CFGs as reasonable or not by comparing them with their corresponding programs. A CFG was considered reasonable if its overall structure was correct, with tolerating missing start or end nodes, a lack of edges to the end node, or sequence statements being stacked in one node. CFGs that incorrectly represented branch and loop structures or fabricated non-existent nodes and edges were labeled as not reasonable. Finally, we counted the number of reasonable CFGs and recorded any issues we identified.

4) Call Graph (CG) Generation: For this task, we randomly selected 16 public source code programs with at least three function calls. These programs, including Python, Java, C, and Solidity, were evaluated using the same method as the AST and CFG generation tasks. A generated call graph (CG) was considered reasonable if all of its call relationships were

[5] https://tree-sitter.github.io/tree-sitter/
[6] https://etherscan.io/

correct, even with some missing calling or redundant nodes. The number of correct CGs generated was recorded.

5) Data Dependency and Taint Analysis: 13 smart contract projects were selected for these tasks. We used Slither [7] with 4.1k starts to extract 992 pairs of data-dependency variables and 830 externally tainted variables. To ensure that both datasets were balanced for each project, we down-sampled them. Each data sample had one of two labels: 1 indicated that it had a data-dependency fact or could be externally tainted, while 0 indicated that it had no data-dependency or could not be externally tainted. We used three performance measurements: Accuracy (ACC), Matthews Correlation Coefficient (MCC), and F1. A higher score for any of these measurements indicated better performance.

6) Pointer Analysis: We collected 40 C programs from Github, which contained 133 pointers. We used Frama-C [21] to extract the possible set of variables for each pointer, which served as the ground truth. Additionally, for each pointer, we collected a set of variables that it may refer to. We used Jaccard index to measure the similarity between the ground truth set and the predicted set. A Jaccard similar coefficient of 1 indicates that the two sets are identical, while a coefficient of 0 indicates that they are completely dissimilar.

7) Equivalent Mutant Detection: We utilized Mutant-Bench [46] for this task. Although the dataset was available in April 2021, all the data information was stored in RDF format, requiring a script to extract the input-label data pairs. As a result, there was a low chance of these input-label data pairs being included in the training dataset of ChatGPT. The dataset consists of 35 programs in total, and we randomly selected 100 equivalent mutants and 100 non-equivalent mutants. We use Recall, Precision, Accuracy and F1 scores in this task. We also check the confusion matrix. Two different prompts were employed for this task: one prompt did not include any example, while the other included demonstration examples.

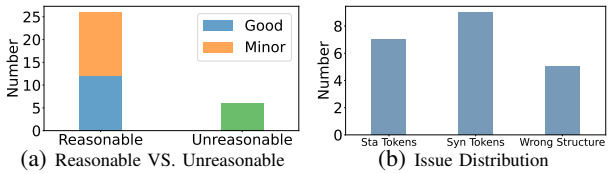
8) Flaky Test Reasoning: We utilized the publicly available FlakyCat dataset [44], which was last updated in August 2022. This dataset, which was manually collected, consists of 13 classes. To create our sample set, we randomly selected 5 samples for each class, resulting in a total of 65 samples. We prompted ChatGPT to assign a label to each input and used Accuracy as the performance metric. Similar to the Equivalent Mutant Detection, we employed two different prompts for this task. We also analyzed the prediction details for each class.

V. Expeimental Results

The generated results for each task by ChatGPT can be found on our website [22]. We provide statistical analysis in the following sections.

A. Code Syntax Understanding (RQ1)

1) AST Generation: Fig. 9a displays the number of reasonable and unreasonable ASTs generated by ChatGPT. In total 32 samples, the majority of the generated ASTs were reasonable, with 26 being reasonable and 6 being unreasonable (represented by the green bar). Of the 26 reasonable ASTs, 12 had no issues (represented by the blue bar), while the remaining ASTs had minor issues (represented by the orange bar).



(a) Reasonable VS. Unreasonable (b) Issue Distribution

Fig. 9: AST Generation Result

We further investigate the issues of the generated ASTs and Fig. 9b displays the number of issues that we identified. A single AST may exhibit multiple issues, and even reasonable ASTs might present some minor issues, as explained below. We categorized the found issues into three groups: missing statement tokens (Sta Tokens), missing syntax tokens (Syn Tokens), and wrong structure. The numbers of issues identified for each category were 7, 9, and 5 as shown in Fig. 9b from left to right, respectively. The Sta Tokens category indicates that some tokens in a statement were missing, such as the token ”System” in ”System.out.print(a);”. The Syn Tokens category indicates that some syntax-related tokens were missing, such as ”public” and ”private” access modifiers. The Wrong Structure category indicates that AST structures were incorrect, such as an incorrect if-else structure. The category Wrong Structure is serious and it means that AST contains faulty syntax structures. Reasonable ASTs with minor issues refer to these ASTs that typically had missing tokens of the statement or missing syntax trivial tokens, such as return type and access modifier. In our evaluation, we find that almost half (4/9) of the missing syntax tokens were related to lacking return types. And half (3/6) of the unreasonable ASTs had incorrect loop structures.

2) Expression Matching: Table II presents the results obtained using ChatGPT. The first column shows the number of ranked equations that were considered in the results. ChatGPT was able to find similar expressions in the top-5 group for 28 out of 32 cases, while CodeBERT only found 8 out of 32 cases. When we consider the top-20 candidates, CodeBERT only found 12 cases. For the 4 cases that were missed by ChatGPT, we manually checked all code files used in the projects and found that these equations did not have corresponding implementations.

In our investigation of the answer explanation details of ChatGPT, we discovered that ChatGPT considers two expressions to be similar if they use similar operators, have similar orders and have a similar number of variables. An example of the target expression and the response from ChatGPT is shown in Fig. 10. The equation above the line represents our target expression, while the response from ChatGPT is shown below the line. We have highlighted the reasons why ChatGPT believes the expressions are similar. We can see that ChatGPT can recognize multiplication and division operators, as well as

[7] https://github.com/crytic/slither

TABLE I: Tasks and Datasets used in this study.

	Level	Language	Evaluation or Comparison	Programs/Projects	Dataset Size
AST	syntax	Python, Java, C, Solidity	Tree-Sitter	32	32
Expression Matching		Solidity	CodeBERT	4	32
CFG	static	Python, Java, C, Solidity	Expert Evaluation	27	27
CG		Python, Java, C, Solidity	Expert Evaluation	16	16
Data Dependence		Solidity	Slither	13	992
Taint Analysis		Solidity	Slither	13	830
Pointer Analysis		C	Frama-C	40	133
Flaky Test Reasoning	dynamic	Java	Expert Evaluation	13	65
Equivalent Mutant Detection		C, Java	Scripts that apply patches	35	200

```
return (liquidate_amount * price_liquidate /
price_collateral_asset) * liquidation_bonus;
```

```
{
"Answer": "Yes",
"similar_expressions": [
{
"function_name": "calculateCollateralAmount",
"line_number": 107,
"expression": "return (tokenPricePerUnit*_liquidateAmount*
liquidationBonus) / collateralPricePerUnit;"
}
],
"Reason": "Both expressions involve multiplication and
division operations with variables representing prices and
bonuses."
}
```

Fig. 10: One Response Example from ChatGPT about Expression Matching.

TABLE II: Expression Matching Results of ChatGPT compared with CodeBERT with 32 total expressions.

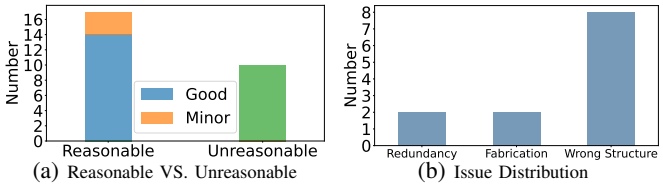
	ChatGPT	CodeBERT
Top-5	28	8
Top-10	28	10
Top-20	28	12
Hit Rate	28/32	12/32

the related operands. An intriguing observation is that, while ChatGPT can identify the line number where the matched expression is located or the starting line number of the function containing the matched expression, none of the line numbers were accurate for these 32 expressions matching samples in either case.

RQ1: In general, ChatGPT is able to understand the syntax structure of the code and the syntax roles of the code tokens. This ability allows it to act as an Abstract Syntax Tree (AST) parser.

B. Code Static Understanding (RQ2)

1) CFG Generation: Fig. 11a shows the number of reasonable and unreasonable CFGs generated by ChatGPT. 14 CFGs (represented by the blue bar) were reasonable without any issues, while 3 CFGs (represented by the orange bar) had minor issues such as two sequence statements in one node, redundant nodes, and missing edges to the exit node. 10 CFGs (represented by the green bar) were incorrect mainly due to wrong loop or if-else structures, and fabrication of non-existent nodes and edges.



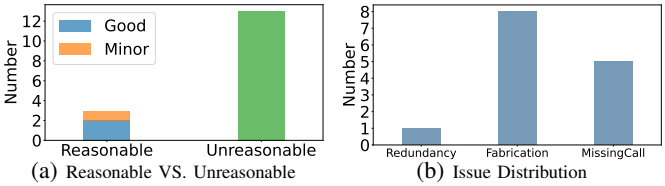
(a) Reasonable VS. Unreasonable (b) Issue Distribution

Fig. 11: CFG Generation Results

Fig. 11b shows the issues we identified, which we categorized into three groups: redundancy, fabrication, and wrong structure. One CFG may have multiple issues and reasonable CFGs only can have the redundancy issue. The redundancy category includes meaningless nodes such as null nodes, while the fabrication category contains non-existent nodes or statements. The wrong structure category refers to CFGs with incorrect structures, such as incorrectly represented loop statements. Redundancy issues are minor because they do not affect the control flow, while fabrication and wrong structure issues are serious because they alter the control flow. We found 2 fabrication issues and 8 wrong structure issues. Of the 8 wrong structure issues, 5 had problems with representing loop control flow, and 2 had problems with representing if-else control flow. Upon examining the identified issues in the AST and CFG generation tasks, an interesting observation is that some serious problems typically arise in relation to loop or if-else statements. ChatGPT appears to have a weaker understanding of the syntax and static behavior of loop and if-else statements.

2) Call Graph Generation: Fig. 12a shows that ChatGPT was unable to generate reasonable CGs for the majority of the samples, with only 3 CGs being reasonable (represented by the blue and orange bars) and 13 being unreasonable (represented by the green bar). Of the 3 reasonable CGs, one had a minor issue containing one redundant null node. Fig. 12b illustrates the issues we found and categorized into 3 groups: redundancy, fabrication, and missing call. The redundancy category contains only one case, which includes a single meaningless null node. The fabrication and missing call categories are more severe and can cause serious problems. We found 8 CG cases where ChatGPT fabricated function calls and 5 CFG cases where ChatGPT did not recognize some function calls.

3) Data Dependency: On the complete dataset, ChatGPT attains ACC 0.69, MCC 0.39, and F1 0.68, as shown in the first row of Table III. This performance is inferior to that of the



(a) Reasonable VS. Unreasonable (b) Issue Distribution

Fig. 12: CG Generation Result

TABLE III: Prediction Performance of ChatGPT on Data Dependence and Taint Analysis on the entire datasets.

	ACC	MCC	F1
Data Dependency	0.69	0.39	0.68
Taint Analysis	0.57	0.20	0.15

static analysis tool Slither. To investigate whether ChatGPT is affected by the data shift issue that causes it to perform differently on various projects, we conducted an analysis. As illustrated in the upper part (marked in blue) of Fig. 13, The F1 score of ChatGPT differs for different projects, with a wide variance ranging from about 0.2 to 1.0. It indicates that ChatGPT cannot perform well on different data distributions (i.e., projects). The performance on different projects can have significant variance. Thus, it suffers from the issue of data shift [47].

4) Taint Analysis: ChatGPT achieved ACC 0.57, MCC 0.20 and F1 0.15 on the entire taint dataset, as presented in the last row of Table III. However, ChatGPT performance is inferior to data dependency analysis, particularly in terms of F1 (please note that we downsampled the taint dataset to make it balanced). To assess whether ChatGPT is suffering from the data shift problem, we conducted an investigation. Our findings indicate that ChatGPT is significantly affected by the data shift problem. The low part (marked in orange) in Fig. 13 depicts the F1 scores of ChatGPT for each program, which display a variation, ranging from 0 to about 0.4.

5) Pointer Analysis: Initially, we determined the number of programs for which ChatGPT fully predicted pointer analysis, and we discovered that it only provided complete predictions for pointers in 3 out of 40 programs. Out of a total of 133 pointer samples from the 40 programs, only 43 were correctly predicted, resulting in an accuracy of about 0.32.

Since one pointer can point to multiple variables, we also computed the Jaccard Index between the predicted set and the ground truth set for each pointer, as displayed in Fig. 14a. We discovered that ChatGPT made completely incorrect predictions for 45 pointers with a correlation of 0. Similarly, for 44 pointers, ChatGPT provided correct predictions for only half. We also computed the average Jaccard index for each program to evaluate whether ChatGPT is affected by the data shift issue for this task. We created a box plot of the mean Jaccard index of pointers from each project, which is illustrated in Fig. 14b. We observed that the index ranges from 0 to 1, with a median value of approximately 0.6. These findings suggest that ChatGPT is indeed affected by the data shift problem.

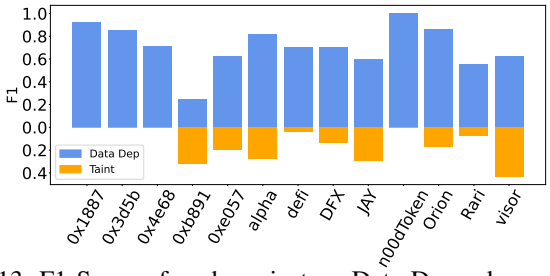
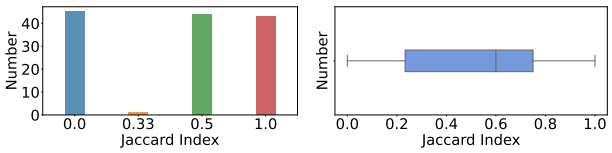


Fig. 13: F1 Score of each project on Data Dependency (blue) and Taint Analysis (orange).



(a) Jaccard Index of Each Pointer (b) Jaccard Index of Each Program

Fig. 14: Jaccard Index of Pointer Analysis.

TABLE IV: Performance about Equivalent Mutant Detection.

	Recall	Precision	ACC	F1
Example	0.46	0.70	0.63	0.55
Non-Example	0.40	0.85	0.67	0.54

TABLE V: Confusion Matrix about Equivalent Mutant detected by ChatGPT including example code in the prompt.

	Predicted Positive	Predicted Negative
Actual Positive	46	54
Actual Negative	20	80

RQ2: ChatGPT has the primary ability to perform code static analysis. However, during the analysis process, ChatGPT experiences the issue of hallucination, which can lead to the fabrication of non-existent elements. Furthermore, the performance of ChatGPT can vary for a given task due to the data shift.

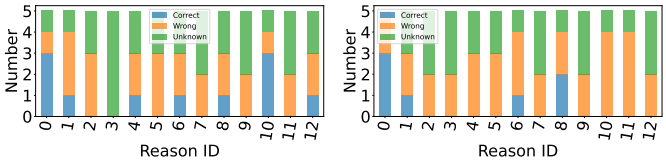
C. Code Dynamic Understanding

1) Equivalent Mutant Detection: Table IV illustrates the performance of ChatGPT in equivalent mutant detection, based on two types of prompts: prompt learning with or without example code. Using example code in the prompt can enhance recall (from 0.40 to 0.46) but reduce precision (from 0.85 to 0.70). This can be interpreted as the equivalent mutant examples guiding ChatGPT to label mutants that are similar to the examples as positive. Since we use multiple diverse examples, ChatGPT tends to assign more positive labels. The confusion matrix for the two types of prompts is displayed in Table VI and Table V. We can observe that ChatGPT predicts 66 positive samples with example code, while the number decreases to 47 without example code. Thus, ChatGPT demonstrates some ability to determine whether a code modification can alter the code behavior in equivalent mutant detection.

2) Flaky Test Reasoning: For this task, we also employed two types of prompts, with or without demonstration example

TABLE VI: Confusion Matrix about Equivalent Mutant detected by ChatGPT excluding example code in the prompt.

	Predicted Positive	Predicted Negative
Actual Positive	40	60
Actual Negative	7	93



(a) prompt without examples (b) prompt with examples

Fig. 15: Predctions of ChatGPT for Flaky Test Reasoning on each category.

code. The task comprises 13 classes, with each class containing five samples. To visualize the predicted label number for each class, we used bar graphs, which are presented in Fig. 15a and Fig. 15b. Fig. 15a depicts the result obtained by excluding example code in the prompt, with an accuracy of 0.17. On the other hand, Fig. 15b illustrates the result obtained by including example code in the prompt, with an accuracy of 0.11. In these figures, the green bar represents the number of predictions that ChatGPT is uncertain about, the orange bar represents the number of incorrect predictions, and the blue bar represents the number of correct predictions. By comparing the blue bars in the two figures, we observed that prompt learning with example code is less effective than prompt learning without example code. Furthermore, by comparing the green bars, we found that ChatGPT with example code is more confident in its responses than without example code for the unsure cases.

RQ3: ChatGPT has limited ability to approximate code dynamic behavior. Few-shot examples increase ChatGPT's confidence in making decisions.

VI. Discussion

A. Limitations

There are several limitations in this study. First, this study does not employ large datasets for analysis as the expensive costs for a large dataset analysis. Second, this study adopts manually designed prompts, however, there are several techniques for automatically designing prompts [48]. It is possible that superior prompts can have better results in this study. Third, since ChatGPT weights are inaccessible, weight analysis commonly used to analyze pre-trained models cannot be utilized in this study [49] and we can only conduct a black analysis. Lastly, as a transformer architecture, ChatGPT imposes a maximum token limitation, restricting the input context in our study.

B. Threats to Validity

Internal Validity: The first threat is that ChatGPT's performance is greatly affected by the designed prompt, to avoid this, we carefully design and optimize the used prompt template. Considering these tasks in our study are domain-related, we add a task and concept explanation in the prompt. We refer to some good prompt engineering repositories [8] to help us design prompts. The second threat is the input length of ChatGPT is limited. To mitigate this problem, we do not use very large programs in the evaluation.

External Validity: ChatGPT is trained using data from the Internet at the end of 2021. Therefore, it is risky to analyze ChatGPT using the current datasets, especially the well-known ones which may mislead us into wrong conclusions. To handle this challenge, we create new knowledge from the raw datasets using the program analysis tools, or use the very recent datasets. These selected program analysis tools are popular, which are widely validated in different applications.

VII. Related Work

Probing Analysis: Probing analysis [50] is a technique employed to examine and interpret the mechanisms of large language models (LLMs). Probing analysis involves designing a probe task, followed by training a probing classifier to predict a property of input tokens based on representations from LLMs. Clark et al. [51] discovered that BERT [52] can encode language syntax by analyzing attention heads. Hewitt et al. [53] proposed a structural probing task to investigate BERT. Betty et al. [54] employed a Question Answering (QA) probing task to analyze the reasoning process of BERT. Eric et al. [55] probed how BERT encodes numeracy. Apidianaki et al. [56] studied how BERT learns word semantics. With the growing popularity of code pre-trained models, Wang et al. [25] and Sergey et al. [57] analyzed how code models learn code syntax using a set of probing tasks

ChatGPT for SE: ChatGPT has garnered increasing interest from the Software Engineering (SE) community due to its impressive performance in general tasks. Researchers in SE have begun exploring the application of ChatGPT to solve SE problems. Xia et al. [10] proposed a dialogue-style approach for automatic program repair. Dominik et al. [23] conducted a bug-fixing study to evaluate ChatGPT's performance. Tian et al. [11] investigated ChatGPT's capabilities in code generation, program repair, and code summarization.

VIII. Conclusion

In this paper, we conduct a comprehensive empirical study to investigate the capabilities of ChatGPT for code analysis. In particular, we study ChatGPT's capability in comprehending code syntax, static behaviors, and dynamic behaviors by **2,327** code samples with **9** different SE tasks. The results of our study indicate that ChatGPT is capable to comprehend code syntax rules and has certain abilities to understand code static behaviors but fail to understand dynamic behaviors. We believe our findings offer insights into ChatGPT's performance on SE tasks and guide the follow-up researchers to effectively utilize ChatGPT in the future.

[8] https://github.com/dair-ai/Prompt-Engineering-Guide/tree/main, https://github.com/f/awesome-chatgpt-prompts

REFERENCES

[1] (2022-11) Chatgpt: Optimizing language models for dialogue. [Online]. Available: https://chat.openai.com

[2] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," *Advances in neural information processing systems*, vol. 30, 2017.

[3] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano, "Learning to summarize with human feedback," *Advances in Neural Information Processing Systems*, vol. 33, pp. 3008–3021, 2020.

[4] J. Wu, L. Ouyang, D. M. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano, "Recursively summarizing books with human feedback," *arXiv preprint arXiv:2109.10862*, 2021.

[5] K. Peng, L. Ding, Q. Zhong, L. Shen, X. Liu, M. Zhang, Y. Ouyang, and D. Tao, "Towards making the most of chatgpt for machine translation," *arXiv preprint arXiv:2303.13780*, 2023.

[6] J. H. Lubowitz, "Chatgpt, an artificial intelligence chatbot, is impacting medical literature," *Arthroscopy*, vol. 39, no. 5, pp. 1121–1122, 2023.

[7] J. Wang, Y. Liang, F. Meng, Z. Li, J. Qu, and J. Zhou, "Cross-lingual summarization via chatgpt," *arXiv preprint arXiv:2302.14229*, 2023.

[8] OpenAI, "Chatgpt demo," https://www.youtube.com/watch?v=outcGtbnMuQ&ab_channel=OpenAI, 2019.

[9] ——, "Gpt-4 technical report," *arXiv*, 2023.

[10] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using chatgpt," *arXiv preprint arXiv:2304.00385*, 2023.

[11] H. Tian, W. Lu, T. O. Li, X. Tang, S.-C. Cheung, J. Klein, and T. F. Bissyandé, "Is chatgpt the ultimate programming assistant–how far is it?" *arXiv preprint arXiv:2304.11938*, 2023.

[12] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," *arXiv preprint arXiv:1711.00740*, 2017.

[13] S. Liu, X. Xie, J. Siow, L. Ma, G. Meng, and Y. Liu, "Graphsearchnet: Enhancing gnns via capturing global dependencies for semantic code search," *IEEE Transactions on Software Engineering*, 2023.

[14] S. Liu, Y. Chen, X. Xie, J. Siow, and Y. Liu, "Retrieval-augmented generation for code summarization via hybrid gnn," *arXiv preprint arXiv:2006.05405*, 2020.

[15] S. Lu, N. Duan, H. Han, D. Guo, S.-w. Hwang, and A. Svyatkovskiy, "Reacc: A retrieval-augmented code completion framework," *arXiv preprint arXiv:2203.07722*, 2022.

[16] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[17] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.

[18] Z. Yang, J. Shi, J. He, and D. Lo, "Natural attack for pre-trained models of code," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1482–1493.

[19] S. Liu, B. Wu, X. Xie, G. Meng, and Y. Liu, "Contrabert: Enhancing code pre-trained models via contrastive learning," *arXiv preprint arXiv:2301.09072*, 2023.

[20] J. Feist, G. Greico, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain*, ser. WETSEB '19. IEEE Press, 2019, p. 8–15. [Online]. Available: https://doi.org/10.1109/WETSEB.2019.00008

[21] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski, "Frama-c," in *Software Engineering and Formal Methods*, G. Eleftherakis, M. Hinchey, and M. Holcombe, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 233–247.

[22] Capabilities of chatgpt for code analysis: An empirical study. [Online]. Available: https://sites.google.com/view/chatgpt4se

[23] D. Sobania, M. Briesch, C. Hanna, and J. Petke, "An analysis of the automatic bug fixing performance of chatgpt," *arXiv preprint arXiv:2301.08653*, 2023.

[24] W. Ma, M. Zhao, X. Xie, Q. Hu, S. Liu, J. Zhang, W. Wang, and Y. Liu, "Is self-attention powerful to learn code syntax and semantics?" *arXiv preprint arXiv:2212.10017*, 2022.

[25] Y. Wan, W. Zhao, H. Zhang, Y. Sui, G. Xu, and H. Jin, "What do they capture? a structural analysis of pre-trained language models for source code," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2377–2388.

[26] J. A. H. López, M. Weyssow, J. S. Cuadrado, and H. Sahraoui, "Ast-probe: Recovering abstract syntax trees from hidden representations of pre-trained language models," *arXiv preprint arXiv:2206.11719*, 2022.

[27] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 783–794.

[28] W. Wang, K. Zhang, G. Li, S. Liu, Z. Jin, and Y. Liu, "A tree-structured transformer for program representation learning," *arXiv preprint arXiv:2208.08643*, 2022.

[29] C. Niu, C. Li, V. Ng, J. Ge, L. Huang, and B. Luo, "Spt-code: Sequence-to-sequence pre-training for learning source code representations," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2006–2018. [Online]. Available: https://doi.org/10.1145/3510003.3510096

[30] X. Jiang, Z. Zheng, C. Lyu, L. Li, and L. Lyu, "Treebert: A tree-based pre-trained model for programming language," in *Uncertainty in Artificial Intelligence*. PMLR, 2021, pp. 54–63.

[31] X. Wang, Y. Wang, F. Mi, P. Zhou, Y. Wan, X. Liu, L. Li, H. Wu, J. Liu, and X. Jiang, "Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation," *arXiv preprint arXiv:2108.04556*, 2021.

[32] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.

[33] W. Ma, M. Zhao, E. Soremekun, Q. Hu, J. M. Zhang, M. Papadakis, M. Cordy, X. Xie, and Y. L. Traon, "Graphcode2vec: generic code embedding via lexical and program dependence analyses," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 524–536.

[34] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," *arXiv preprint arXiv:2103.06333*, 2021.

[35] X. Wang, Y. Wang, Y. Wan, J. Wang, P. Zhou, L. Li, H. Wu, and J. Liu, "Code-mvp: learning to represent source code from multiple views with contrastive pre-training," *arXiv preprint arXiv:2205.02029*, 2022.

[36] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[37] H. Ye, M. Martinez, and M. Monperrus, "Neural program repair with execution-based backpropagation," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1506–1518. [Online]. Available: https://doi.org/10.1145/3510003.3510222

[38] X. Jin, K. Pei, J. Y. Won, and Z. Lin, "Symlm: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1631–1645. [Online]. Available: https://doi.org/10.1145/3548606.3560612

[39] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "Trex: Learning execution semantics from micro-traces for binary similarity," *arXiv preprint arXiv:2012.08680*, 2020.

[40] X. Cheng, H. Wang, J. Hua, M. Zhang, G. Xu, L. Yi, and Y. Sui, "Static detection of control-flow-related vulnerabilities using graph embedding," in *2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS)*. IEEE, 2019, pp. 41–50.

[41] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Trans. Program. Lang. Syst.*, vol. 9, no. 3, p. 319–349, jul 1987. [Online]. Available: https://doi.org/10.1145/24039.24041

[42] F. E. Allen, "Control flow analysis," in *Proceedings of a Symposium on Compiler Optimization*. New York, NY, USA: Association for Computing Machinery, 1970, p. 1–19. [Online]. Available: https://doi.org/10.1145/800028.808479

[43] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: an analysis and survey," in *Advances in Computers*. Elsevier, 2019, vol. 112, pp. 275–378.

[44] A. Akli, G. Haben, S. Habchi, M. Papadakis, and Y. L. Traon, "Predicting flaky tests categories using few-shot learning," *arXiv preprint arXiv:2208.14799*, 2022.

[45] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[46] L. van Hijfte and A. Oprescu, "Mutantbench: an equivalent mutant problem comparison framework," in *2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2021, pp. 7–12.

[47] B. Turhan, "On the dataset shift problem in software engineering prediction models," *Empirical Softw. Engg.*, vol. 17, no. 1–2, p. 62–74, feb 2012. [Online]. Available: https://doi.org.remotexs.ntu.edu.sg/10.1007/s10664-011-9182-8

[48] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, "Autoprompt: Eliciting knowledge from language models with automatically generated prompts," *arXiv preprint arXiv:2010.15980*, 2020.

[49] A. Rogers, O. Kovaleva, and A. Rumshisky, "A Primer in BERTology: What We Know About How BERT Works," *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 842–866, 01 2021. [Online]. Available: https://doi.org/10.1162/tacl_a_00349

[50] ——, "A primer in bertology: What we know about how bert works," *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 842–866, 2021.

[51] K. Clark, U. Khandelwal, O. Levy, and C. D. Manning, "What does bert look at? an analysis of bert's attention," *arXiv preprint arXiv:1906.04341*, 2019.

[52] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[53] J. Hewitt and C. D. Manning, "A structural probe for finding syntax in word representations," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4129–4138. [Online]. Available: https://aclanthology.org/N19-1419

[54] B. van Aken, B. Winter, A. Löser, and F. A. Gers, "How does bert answer questions? a layer-wise analysis of transformer representations," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, ser. CIKM '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1823–1832. [Online]. Available: https://doi.org/10.1145/3357384.3358028

[55] E. Wallace, Y. Wang, S. Li, S. Singh, and M. Gardner, "Do nlp models know numbers? probing numeracy in embeddings," *arXiv preprint arXiv:1909.07940*, 2019.

[56] M. Apidianaki and A. G. Soler, "All dolphins are intelligent and some are friendly: Probing bert for nouns' semantic properties and their prototypicality," *arXiv preprint arXiv:2110.06376*, 2021.

[57] S. Troshin and N. Chirkova, "Probing pretrained models of source codes," in *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*. Abu Dhabi, United Arab Emirates (Hybrid): Association for Computational Linguistics, Dec. 2022, pp. 371–383. [Online]. Available: https://aclanthology.org/2022.blackboxnlp-1.31